

SQL Server — Guia de Estudo Completo

Plano de 2 horas · do básico ao avançado · tudo para fazer qualquer nível de consulta

Roteiro (2h)

- 1. SELECT, JOIN e filtros — 15 min
- 2. Agregação & GROUP BY — 10 min
- 3. Subqueries & CTE — 15 min
- 4. Window Functions — 30 min ★
- 5. Views — 15 min
- 6. Stored Procedures — 20 min
- 7. Functions (UDF) — 10 min
- 8. Triggers — 15 min
- 9. Cheat sheet final

1. Fundamentos de consulta

15 min

O que é: a base de tudo. **SELECT** lê dados; **JOIN** combina tabelas relacionadas; **WHERE** filtra linhas antes de agrupar. A ordem lógica de execução é: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

BÁSICO SELECT + WHERE

```
-- colunas específicas, filtro e ordenação
SELECT Nome, Email, Cidade
FROM Cliente
WHERE Cidade = 'João Pessoa' AND Ativo = 1
ORDER BY Nome ASC;
```

MÉDIO JOINS entre tabelas

```
-- INNER: só quem tem correspondência nas duas tabelas
SELECT p.Id, c.Nome, p.Valor
FROM Pedido p
INNER JOIN Cliente c ON c.Id = p.ClienteId;

-- LEFT: traz TODOS os clientes, mesmo sem pedido (NULL no pedido)
SELECT c.Nome, p.Valor
FROM Cliente c
LEFT JOIN Pedido p ON p.ClienteId = c.Id;
```

AVANÇADO Filtros condicionais e múltiplos JOINS

```
-- CASE no SELECT + filtro com EXISTS
SELECT c.Nome,
       CASE WHEN SUM(p.Valor) > 5000 THEN 'VIP'
            WHEN SUM(p.Valor) > 1000 THEN 'Médio'
            ELSE 'Comum' END AS Faixa
FROM Cliente c
JOIN Pedido p ON p.ClienteId = c.Id
JOIN ItemPedido i ON i.PedidoId = p.Id
WHERE EXISTS (SELECT 1 FROM Pagamento pg WHERE pg.PedidoId = p.Id)
GROUP BY c.Nome;
```

Atenção: JOIN com tabela que tem muitas linhas **multiplica** o resultado antes do GROUP BY (cuidado ao somar valores — pode duplicar).

2. Agregação & GROUP BY

10 min

O que é: funções que **colapsam várias linhas em uma** (COUNT, SUM, AVG, MIN, MAX). **GROUP BY** define os grupos; **HAVING** filtra os grupos já agregados (o WHERE não consegue).

BÁSICO

```
SELECT COUNT(*) AS Total, AVG(Valor) AS Media
FROM Pedido;
```

MÉDIO GROUP BY + HAVING

```
SELECT ClienteId, SUM(Valor) AS Total
FROM Pedido
GROUP BY ClienteId
HAVING SUM(Valor) > 1000 -- filtra grupos, não linhas
ORDER BY Total DESC;
```

AVANÇADO Agregação condicional (pivô manual)

```
-- conta status diferentes em colunas separadas
SELECT ClienteId,
    SUM(CASE WHEN Status = 'Pago' THEN 1 ELSE 0 END) AS Pagos,
    SUM(CASE WHEN Status = 'Pendente' THEN 1 ELSE 0 END) AS Pendentes
FROM Pedido
GROUP BY ClienteId;
```

3. Subqueries & CTE

15 min

O que é: **Subquery** = consulta dentro de outra. **CTE** (WITH ... AS) = subquery nomeada e legível, declarada antes do SELECT. É o que permite **filtrar por window functions** (não dá pra usar ROW_NUMBER() no WHERE direto).

BÁSICO Subquery escalar

```
SELECT Nome, Valor
FROM Pedido
WHERE Valor > (SELECT AVG(Valor) FROM Pedido);
```

MÉDIO CTE simples

```
WITH Vendas AS (
    SELECT ClienteId, SUM(Valor) AS Total
    FROM Pedido GROUP BY ClienteId
)
SELECT c.Nome, v.Total
FROM Vendas v JOIN Cliente c ON c.Id = v.ClienteId
WHERE v.Total > 1000;
```

AVANÇADO CTE recursiva (hierarquia / organograma)

```
WITH Hierarquia AS (  
  -- âncora: o topo da árvore  
  SELECT Id, Nome, GerenteId, 0 AS Nivel  
  FROM Funcionario WHERE GerenteId IS NULL  
  UNION ALL  
  -- parte recursiva: desce nível a nível  
  SELECT f.Id, f.Nome, f.GerenteId, h.Nivel + 1  
  FROM Funcionario f  
  JOIN Hierarquia h ON f.GerenteId = h.Id  
)  
SELECT REPLICATE(' ', Nivel) + Nome AS Arvore, Nivel  
FROM Hierarquia ORDER BY Nivel;
```

Dica: CTE recursiva precisa de âncora + UNION ALL + referência a si mesma. Limite padrão de 100 níveis (mude com OPTION (MAXRECURSION n)).

4. Window Functions

30 min ★

O que é (o mais importante): funções que calculam sobre uma "janela" de linhas **sem colapsar** o resultado — cada linha mantém sua identidade mas ganha um cálculo do grupo. Sintaxe: FUNÇÃO() OVER (PARTITION BY ... ORDER BY ...).

Categoria	Funções	Uso
Ranking	ROW_NUMBER, RANK, DENSE_RANK, NTILE	posição / top N por grupo
Agregação	SUM, AVG, COUNT, MIN, MAX OVER	total do grupo em cada linha
Offset	LAG, LEAD, FIRST_VALUE, LAST_VALUE	comparar com linha anterior/próxima

BÁSICO Numerar e somar sem agrupar

```
SELECT Nome, Departamento, Salario,  
  ROW_NUMBER() OVER (ORDER BY Salario DESC) AS Posicao,  
  SUM(Salario) OVER () AS TotalGeral  
FROM Funcionario;
```

MÉDIO PARTITION BY + ranking por grupo

```
SELECT Nome, Departamento, Salario,  
  ROW_NUMBER() OVER (PARTITION BY Departamento ORDER BY Salario DESC) AS rn,  
  RANK() OVER (PARTITION BY Departamento ORDER BY Salario DESC) AS rk,  
  DENSE_RANK() OVER (PARTITION BY Departamento ORDER BY Salario DESC) AS drk,  
  AVG(Salario) OVER (PARTITION BY Departamento) AS MediaDepto  
FROM Funcionario;
```

Diferença: **ROW_NUMBER** nunca repete (1,2,3,4). **RANK** pula após empate (1,1,3). **DENSE_RANK** não pula (1,1,2).

AVANÇADO Top-N por grupo, LAG/LEAD e acumulado com frame

```
-- top 3 salários por departamento (precisa de CTE)
WITH R AS (
  SELECT *, ROW_NUMBER() OVER
    (PARTITION BY Departamento ORDER BY Salario DESC) AS rn
  FROM Funcionario
)
SELECT * FROM R WHERE rn <= 3;

-- comparar com mês anterior + total acumulado (frame ROWS)
SELECT Mes, Faturamento,
  LAG(Faturamento) OVER (ORDER BY Mes) AS MesAnterior,
  Faturamento - LAG(Faturamento) OVER (ORDER BY Mes) AS Variacao,
  SUM(Faturamento) OVER (ORDER BY Mes
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS Acumulado,
  AVG(Faturamento) OVER (ORDER BY Mes
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS MediaMovel3
FROM FaturamentoMensal;
```

Armadilha clássica: LAST_VALUE sem frame retorna a linha atual, não o último do grupo. Use ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

5. Views

15 min

O que é: uma consulta salva com nome, usada como se fosse uma tabela. Não armazena dados (exceto a indexed view) — só guarda a definição. Serve para reaproveitar lógica e simplificar/segurar acesso.

BÁSICO

```
CREATE VIEW vw_ClientesAtivos AS
SELECT Id, Nome, Email FROM Cliente WHERE Ativo = 1;

SELECT * FROM vw_ClientesAtivos; -- usa como tabela
```

MÉDIO View com JOIN e agregação

```
CREATE VIEW vw_VendasPorCliente AS
SELECT c.Id, c.Nome, SUM(p.Valor) AS Total, COUNT(*) AS Qtd
FROM Cliente c
JOIN Pedido p ON p.ClienteId = c.Id
GROUP BY c.Id, c.Nome;

SELECT * FROM vw_VendasPorCliente WHERE Total > 1000;
```

AVANÇADO Indexed view (materializada, ganho de performance)

```
CREATE VIEW vw_ResumoVendas WITH SCHEMABINDING AS
SELECT ClienteId, SUM(Valor) AS Total, COUNT_BIG(*) AS Qtd
FROM dbo.Pedido -- schema obrigatório com SCHEMABINDING
GROUP BY ClienteId;
GO

-- o índice materializa o resultado em disco
CREATE UNIQUE CLUSTERED INDEX ix_Resumo
ON vw_ResumoVendas(ClienteId);
```

Regras: view não aceita ORDER BY (a menos que tenha TOP). Indexed view exige SCHEMABINDING, COUNT_BIG(*) e nomes com schema (dbo.). ALTER VIEW muda a definição.

6. Stored Procedures

20 min

O que é: bloco de código SQL salvo no banco, executável por nome, com **parâmetros** de entrada/saída. Encapsula lógica, melhora performance (plano em cache) e segurança. Pode ter variáveis, fluxo (IF/WHILE), transações e tratamento de erro.

BÁSICO

```
CREATE PROCEDURE sp_ListarClientes
AS
BEGIN
    SET NOCOUNT ON;
    SELECT Id, Nome FROM Cliente ORDER BY Nome;
END;

EXEC sp_ListarClientes;
```

MÉDIO Parâmetros de entrada e saída

```
CREATE PROCEDURE sp_TotalCliente
    @ClienteId INT,
    @DataInicio DATE = NULL,          -- opcional
    @Total MONEY OUTPUT               -- saída
AS
BEGIN
    SET NOCOUNT ON;
    SELECT @Total = SUM(Valor)
    FROM Pedido
    WHERE ClienteId = @ClienteId
    AND (@DataInicio IS NULL OR Data >= @DataInicio);
END;

DECLARE @t MONEY;
EXEC sp_TotalCliente @ClienteId = 5, @Total = @t OUTPUT;
PRINT @t;
```

AVANÇADO Transação + TRY/CATCH

```
CREATE PROCEDURE sp_TransferirSaldo
    @De INT, @Para INT, @Valor MONEY
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        UPDATE Conta SET Saldo = Saldo - @Valor WHERE Id = @De;
        IF (SELECT Saldo FROM Conta WHERE Id = @De) < 0
            THROW 50001, 'Saldo insuficiente', 1;
        UPDATE Conta SET Saldo = Saldo + @Valor WHERE Id = @Para;
        COMMIT;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        THROW; -- repassa o erro original
    END CATCH;
END;
```

SET NOCOUNT ON evita mensagens de "linhas afetadas". @@TRANCOUNT checa se há transação aberta antes do ROLLBACK.

7. Functions (UDF)

10 min

O que é: função definida pelo usuário que **retorna um valor ou tabela** e pode ser usada dentro de SELECT/ WHERE (diferente da procedure, que não retorna direto numa consulta). Tipos: **escalar** (1 valor) e **table-valued** (tabela).

BÁSICO Função escalar

```
CREATE FUNCTION fn_NomeCompleto(@nome VARCHAR(50), @sob VARCHAR(50))
RETURNS VARCHAR(101)
AS
BEGIN
    RETURN @nome + ' ' + @sob;
END;

SELECT dbo.fn_NomeCompleto(Nome, Sobrenome) FROM Cliente;
```

AVANÇADO Inline table-valued function (a mais performática)

```
CREATE FUNCTION fn_PedidosCliente(@id INT)
RETURNS TABLE
AS
RETURN (
    SELECT Id, Data, Valor
    FROM Pedido WHERE ClienteId = @id
);

-- usa como tabela, inclusive com CROSS APPLY
SELECT c.Nome, p.Valor
FROM Cliente c
CROSS APPLY dbo.fn_PedidosCliente(c.Id) p;
```

Evite funções escalares dentro de WHERE em tabelas grandes — executam linha a linha e matam performance. Prefira **inline TVF**.

8. Triggers

15 min

O que é: código que **dispara automaticamente** em resposta a INSERT, UPDATE ou DELETE numa tabela. Usado para auditoria, validação e regras de negócio. Dentro do trigger há duas tabelas virtuais: **inserted** (linhas novas) e **deleted** (linhas antigas).

Operação	inserted	deleted
INSERT	✓ linhas novas	vazia
DELETE	vazia	✓ linhas removidas
UPDATE	✓ valores novos	✓ valores antigos

BÁSICO Auditoria de INSERT

```
CREATE TRIGGER trg_LogPedido
ON Pedido
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO LogPedido(PedidoId, Data)
    SELECT Id, GETDATE() FROM inserted;
END;
```

MÉDIO Auditoria de UPDATE (antes x depois)

```
CREATE TRIGGER trg_AuditSalario
ON Funcionario
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditSalario(FuncId, SalarioAntigo, SalarioNovo, Data)
    SELECT i.Id, d.Salario, i.Salario, GETDATE()
    FROM inserted i
    JOIN deleted d ON d.Id = i.Id
    WHERE i.Salario <> d.Salario; -- só se mudou
END;
```

AVANÇADO INSTEAD OF para bloquear/validar

```
-- intercepta o DELETE: em vez de apagar, inativa (soft delete)
CREATE TRIGGER trg_NaoApagarCliente
ON Cliente
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE Cliente SET Ativo = 0
    WHERE Id IN (SELECT Id FROM deleted);
END;
```

Regra de ouro: trigger dispara **uma vez por comando**, não por linha. Sempre trate inserted/deleted como **conjuntos** (com JOIN/IN), nunca assumo 1 linha. AFTER = depois da operação; INSTEAD OF = substitui a operação.

9. Cheat sheet final

Quero...	Uso
Numerar/ranquear linhas	ROW_NUMBER / RANK / DENSE_RANK OVER
Top N por grupo	CTE + ROW_NUMBER + WHERE rn <= N
Total/média sem perder linhas	SUM/AVG OVER (PARTITION BY)
Comparar com linha anterior	LAG / LEAD
Total acumulado / média móvel	SUM OVER (ORDER BY ... ROWS BETWEEN ...)
Filtrar grupo agregado	HAVING
Hierarquia / árvore	CTE recursiva (UNION ALL)
Reaproveitar consulta	VIEW
Lógica com parâmetros + transação	STORED PROCEDURE
Retornar valor/tabela em SELECT	FUNCTION (escalar / inline TVF)
Reagir a INSERT/UPDATE/DELETE	TRIGGER (inserted / deleted)
Pivô / contagem condicional	SUM(CASE WHEN ...)

Ordem de estudo recomendada: domine 1 → 2 → 3 (base de qualquer consulta), invista pesado no bloco 4 (window functions resolvem 80% das consultas "difíceis"), e trate 5 → 8 como objetos do banco para automação e organização.